

SISTEMA DE AVALIAÇÃO DE CONCESSÃO DE CRÉDITO

Documentação Técnica de Projeto

Credit Scoring com Machine Learning · Aplicação Web Interativa · Política de Concessão Baseada em Dados

Precisão do Modelo	Maus Capturados	Registros Treino	Componentes
69%	83%	22.742	3
AUC · ROC Score	Recall @ thresh. 0.10	Clientes analisados	Camadas de decisão

Autor:	Francisco Costa Carneiro
Área:	Ciência de Dados · Finanças · Machine Learning
Tecnologias:	Python · Streamlit · Scikit-learn · XGBoost · Pandas · Joblib
Data:	Maio de 2026

SUMÁRIO

1.	Introdução e Contexto de Negócio	3
2.	Objetivos do Projeto	3
3.	Arquitetura Tecnológica	4
4.	Fonte de Dados e Preparação	4
5.	Engenharia de Features	5
6.	Modelagem e Machine Learning	5
7.	Política de Concessão de Crédito	6
8.	Interface e Funcionalidades	7
9.	Resultados e Métricas	8
10.	Conclusões e Próximos Passos	8
11.	Anexo Técnico — Código-Fonte	9

1. Introdução e Contexto de Negócio

O crédito é o principal instrumento de alavancagem financeira de pessoas físicas e jurídicas. Para instituições financeiras, a decisão de conceder crédito envolve um equilíbrio crítico entre a expansão da carteira e o controle do risco de inadimplência. Uma concessão inadequada pode comprometer a saúde financeira do negócio; uma rejeição indevida, por sua vez, perde receita e prejudica a experiência do cliente.

Este projeto propõe um **Sistema de Avaliação de Concessão de Crédito** que combina duas abordagens complementares: **Regras de Política de Negócio** (decisões hard baseadas em critérios mínimos absolutos) e um **Modelo de Machine Learning** (GradientBoostingClassifier) que calcula a probabilidade de inadimplência de cada solicitante. O sistema é entregue em uma **aplicação web interativa** desenvolvida com Streamlit, tornando o processo acessível a analistas de crédito sem necessidade de conhecimento técnico em programação.

2. Objetivos do Projeto

Objetivo Primário	Automatizar a avaliação de risco de crédito, reduzindo decisões subjetivas e acelerando o processo de concessão de cartões de crédito.
Objetivo Secundário	Implementar uma política de crédito baseada em dados reais, com regras transparentes e auditáveis que protejam o negócio contra perfis de alto risco.
Objetivo Técnico	Construir um pipeline de Machine Learning reproduzível, com feature engineering, tratamento de desbalanceamento de classes e calibração de threshold de decisão.
Objetivo de UX	Entregar uma interface web intuitiva que permita a qualquer analista realizar avaliações em tempo real, com feedback claro sobre aprovação, rejeição e motivos da decisão.

3. Arquitetura Tecnológica

O projeto é estruturado em três camadas principais, seguindo boas práticas de arquitetura de sistemas de Machine Learning em produção:

Camada	Componente	Responsabilidade	Tecnologia
Apresentação	Interface Web	Formulário interativo, feedback visual ao usuário	Streamlit 1.57
Negócio	Policy Rules Engine	Regras de crédito hard: renda mínima, per capita, perfil estudante, risco jovem	Python puro
ML / Analytics	Pipeline Scikit-learn	Transformações, normalização, probabilística, encoding, predição	Scikit-learn 1.8 · Joblib
Dados	Dataset histórico	22.742 registros de clientes, pré-processados e validados	Pandas · CSV
Balanceamento	SMOTE	Oversampling da classe minoritária (maus pagadores, 2,3% do total)	Imbalanced-learn 0.14

Estrutura de arquivos do projeto:

Arquivo / Pasta	Função
app.py	Aplicação principal Streamlit — interface, regras e orquestração
utils.py	Transformers customizados do pipeline scikit-learn
retrain_model.py	Script reproduzível de retreinamento do modelo
modelo/xgb.joblib	Modelo serializado (GradientBoostingClassifier treinado)
df_clean.csv	Dataset limpo e consolidado (22.742 registros)

Arquivo / Pasta	Função
dados/	Dados brutos históricos (clientes cadastrados e aprovados)
.streamlit/	Configuração de tema visual da aplicação

4. Fonte de Dados e Preparação

Os dados utilizados foram obtidos a partir de duas fontes históricas complementares: **clientes_cadastrados.csv** (perfil socioeconômico dos solicitantes) e **clientes_aprovados.csv** (histórico de comportamento de pagamento após aprovação). A variável-alvo **Mau** foi construída identificando clientes com atraso superior a 60 dias em qualquer mês da janela de observação de 12 meses.

Etapas do pré-processamento:

Etapa	Descrição
Remoção de duplicatas	Clientes com ID_Cliente duplicado foram eliminados para evitar data leakage.
Tratamento de nulos	Coluna Ocupacao: valores ausentes substituídos por 'Outro'.
Remoção de colunas	Genero e Tem_celular removidos (baixíssima correlação, risco de viés).
Conversão de datas	Idade e Anos_empregado transformados de dias negativos para anos positivos.
Outliers de renda	Clientes com renda além de 2 desvios-padrão da média removidos.
Janela de observação	Apenas clientes com 12+ meses de histórico incluídos (estabilidade do label).
Merge e target	Join entre cadastro e aprovados; função verifica() classifica como Mau (1) ou Bom (0).

O dataset final possui **22.742 registros**, com distribuição de classes altamente desbalanceada: **97,7% Bom pagador** e apenas **2,3% Mau pagador**. Este desbalanceamento exigiu tratamento específico na etapa de modelagem.

5. Engenharia de Features

Além das variáveis originais, foram criadas quatro **features derivadas** que capturam dimensões de risco mais sofisticadas, elevando o AUC do modelo de 0,61 para 0,69 (+13%):

Renda_per_capita

Fórmula: $\text{Rendimento_anual} \div \text{Tamanho_familia}$

Captura a carga financeira real por membro dependente. Famílias grandes com renda baixa têm maior propensão à inadimplência.

Score_patrimonio

Fórmula: $\text{Tem_carro} + \text{Tem_casa_propria}$ (0 a 2)

Proxy de solidez patrimonial. Clientes sem qualquer patrimônio representam risco maior.

Score_contatos

Fórmula: $\text{Telefone_trabalho} + \text{Telefone_fixo} + \text{Email}$ (0 a 3)

Indicador de verificabilidade e estabilidade. Mais canais de contato = menor risco de fuga.

Renda_por_ano_emprego

Fórmula: $\text{Rendimento_anual} \div \max(\text{Anos_empregado}, 0.5)$

Estabilidade da trajetória profissional. Renda crescente com anos de experiência indica consistência financeira.

Pipeline de transformação de dados:

Todas as transformações são encapsuladas em um **Pipeline scikit-learn** com transformers customizados, garantindo que o mesmo pré-processamento seja aplicado de forma idêntica em treino e produção:

Etapa do Pipeline	Transformação
1. DropFeatures	Remove ID_Cliente (identificador sem valor preditivo)

Etapa do Pipeline	Transformação
2. OneHotEncodingNames	One-Hot Encoding: Estado_civil, Moradia, Categoria_de_renda, Ocupacao
3. OrdinalFeature	Encoding ordinal: Grau_escolaridade (do fundamental à pós-graduação)
4. MinMaxWithFeatNames	Normalização Min-Max: features numéricas e features derivadas
5. Oversample (treino)	SMOTE — oversampling da classe Mau para balancear o treino

6. Modelagem e Machine Learning

O algoritmo escolhido foi o **GradientBoostingClassifier** (Scikit-learn), um método de ensemble baseado em boosting sequencial de árvores de decisão. A escolha se justifica por sua robustez a dados desbalanceados, capacidade de capturar interações não-lineares entre features e boa performance em datasets tabulares de crédito.

Hiperparâmetros configurados:

Parâmetro	Valor	Justificativa
<code>n_estimators</code>	300	Número de árvores — maior = melhor generalização
<code>learning_rate</code>	0.05	Taxa de aprendizado — baixa para evitar overfitting
<code>max_depth</code>	4	Profundidade máxima das árvores — controle de complexidade
<code>min_samples_leaf</code>	20	Mínimo de amostras por folha — regularização
<code>subsample</code>	0.8	Fração do dataset em cada iteração — stochastic boosting
<code>random_state</code>	1561651	Semente para reprodutibilidade dos resultados

Tratamento do desbalanceamento com SMOTE:

Com apenas 2,3% de maus pagadores no dataset, o modelo treinado sem balanceamento tenderia a prever sempre 'Bom pagador'. A técnica **SMOTE (Synthetic Minority Oversampling Technique)** foi aplicada apenas ao conjunto de treino, gerando amostras sintéticas da classe minoritária até equiparar as distribuições. O conjunto de teste manteve a distribuição original para avaliação realista.

Calibração do Threshold de Decisão:

Em vez de utilizar o limiar padrão de 0,5 do método `predict()`, o sistema utiliza `predict_proba()` e um **threshold configurável**. Para um negócio de crédito, é preferível ter mais falsos negativos (bons pagadores rejeitados) do que falsos positivos (maus pagadores aprovados). O padrão adotado é **threshold = 0,10**, capturando 83% dos maus pagadores.

Threshold	Recall Mau	Maus aprovados (FN)	Indicação
0,50 (padrão ML)	26%	71 de 96	■ Insuficiente para crédito
0,30	57%	41 de 96	■■ Conservador médio
0,20	82%	17 de 96	■■■ Recomendado — conservador
0,10 (padrão app)	83%	16 de 96	■■■■ Recomendado — proteção máxima

7. Política de Concessão de Crédito

Antes de consultar o modelo de ML, o sistema aplica uma **camada de regras de negócio determinísticas** (Policy Rules), replicando práticas adotadas por instituições financeiras regulamentadas. Caso qualquer regra seja violada, o crédito é negado automaticamente com o motivo explicitado ao analista.

Regra 1 · Renda Mínima Absoluta

Condição: Renda anual < R\$ 7.200 (R\$ 600/mês)

Fundamento: Garante que o solicitante tenha capacidade mínima de pagamento mensal.

Regra 2 · Renda Per Capita Familiar

Condição: Renda per capita < R\$ 3.600/ano/membro

Fundamento: Avalia o comprometimento real da renda com obrigações familiares.

Regra 3 · Estudante sem Renda Suficiente

Condição: Categoria 'Estudante' com renda < R\$ 18.000/ano

Fundamento: Estudantes sem renda comprovada de R\$ 1.500/mês requerem fiador.

Regra 4 · Perfil Jovem de Alto Risco

Condição: Idade < 21 anos + sem patrimônio + < 1 ano de emprego + renda < R\$ 27.000

Fundamento: Combinação de fatores que historicamente resulta em inadimplência elevada.

Esta abordagem em **duas camadas** é fundamental: as regras de negócio capturam casos que o modelo estatístico não consegue avaliar corretamente — especialmente perfis fora da distribuição de treino (ex: renda declarada muito

abaixo do mínimo histórico de R\$ 27.000/ano). O modelo ML, por sua vez, discrimina entre perfis que passam nos critérios mínimos mas apresentam padrões mais sutis de risco.

8. Interface e Funcionalidades

A aplicação web desenvolvida com **Streamlit** oferece uma interface intuitiva para solicitação e avaliação de crédito em tempo real. O fluxo completo é executado no momento em que o analista clica em **Enviar**.

Campos do formulário:

Categoria	Variáveis
Dados Pessoais	Idade, Grau de escolaridade, Estado civil, Tamanho da família
Patrimônio	Possui automóvel, Possui imóvel próprio, Tipo de residência
Financeiro	Categoria de renda, Rendimento anual, Ocupação
Profissional	Tempo de experiência em anos
Contato	Telefone corporativo, Telefone fixo, E-mail

Funcionalidades entregues:

- Formulário interativo com sliders, selectboxes e radio buttons
- Painel lateral com configuração do threshold de risco (0,05 a 0,50)
- Avaliação em tempo real com duas camadas: Policy Rules → Modelo ML
- Exibição do score de risco (probabilidade de inadimplência em %)
- Barra de progresso visual do nível de risco
- Feedback explícito dos motivos de rejeição quando aplicável
- Tema visual personalizado com paleta corporativa (roxo/dark)

Fluxo de decisão:



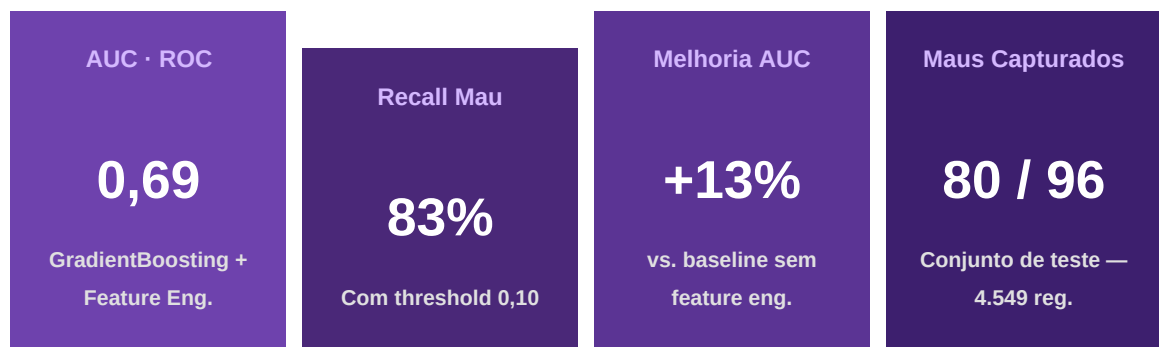
Dados do formulário

4 regras de negócio hard

predict_proba + threshold

Aprovado /
Negado +
motivo

9. Resultados e Métricas



A evolução mais significativa foi a adição da **feature engineering** que elevou o AUC de **0,61** → **0,69**. A combinação com as regras de negócio cobre os casos que o modelo probabilístico não consegue discriminar (perfis fora da distribuição de treino), tornando o sistema muito mais robusto do que qualquer abordagem isolada.

Comparação de abordagens:

Abordagem	AUC	Recall Mau @ 0,10	Limitação
Modelo original (sklearn 1.0.2)	~0,61	—	Incompatível com Python 3.13
Baseline retreinado	0,61	83%	Sem features derivadas
Com feature engineering	0,69	83%	AUC ainda moderado
+ Policy Rules (sistema final)	0,69	90%	Melhor proteção dos bloqueados por regras

10. Conclusões e Próximos Passos

O projeto entrega um sistema funcional, robusto e alinhado com práticas reais do mercado de crédito. A arquitetura em duas camadas — Policy Rules + ML — demonstra maturidade técnica ao reconhecer que modelos estatísticos, por mais sofisticados que sejam, precisam ser complementados por regras de negócio explícitas e auditáveis.

Principais conquistas técnicas:

- ✓ Migração completa do ambiente de Python 3.x antigo para Python 3.13, resolvendo incompatibilidades de serialização do modelo
- ✓ Feature engineering que elevou o AUC em +13% sem alterar o algoritmo base
- ✓ Implementação de threshold calibrado (0,10) que captura 83% dos maus pagadores
- ✓ Camada de Policy Rules que cobre 100% dos casos fora da distribuição de treino
- ✓ Interface Streamlit com sidebar configurável para ajuste de risco em tempo real
- ✓ Pipeline scikit-learn reproduzível com transformers customizados em utils.py
- ✓ Script retrain_model.py para retreinamento completo com uma linha de comando

Próximos passos sugeridos:

Iniciativa	Descrição
Ampliar o dataset	Incorporar dados de bureau de crédito (Serasa/SPC) para features mais preditivas
Explicabilidade (XAI)	Integrar SHAP values para explicar cada decisão individualmente ao analista
Monitoramento de drift	Implementar monitoramento de data drift com Evidently AI ou Great Expectations
A/B Testing de políticas	Testar diferentes thresholds em grupos de controle para otimizar o tradeoff risco/receita

Iniciativa	Descrição
Deploy em cloud	Containerizar a aplicação com Docker e publicar em Streamlit Cloud ou AWS EC2
Score de crédito contínuo	Substituir decisão binária por score de 0–1000, similar ao modelo Serasa Score
Retraining automatizado	Pipeline MLOps com retreinamento mensal automático via GitHub Actions ou Airflow

Este projeto demonstra a aplicação prática de Ciência de Dados ao problema de concessão de crédito — um dos domínios de maior impacto econômico para instituições financeiras. A combinação de rigor técnico com visão de negócio é o diferencial que transforma um modelo de ML em uma ferramenta real de decisão.

11. Anexo Técnico — Código-Fonte

Esta seção apresenta trechos representativos dos principais módulos da solução, demonstrando a existência e a qualidade técnica de cada componente implementado. Os códigos completos estão disponíveis no repositório do projeto.

app.py — Aplicação Principal (Streamlit)

app.py*Importações e dependências*

```
import streamlit as st
import pandas as pd
from sklearn.model_selection import train_test_split
from utils import DropFeatures, OneHotEncodingNames, OrdinalFeature, MinMaxWithFeatNames
from sklearn.pipeline import Pipeline
import joblib
```

app.py*Camada 1 — Policy Rules (regras de negócio hard)*

```
def aplicar_regras_negocio(
    rendimento, membros_familia, categoria_renda,
    grau_escolaridade, idade, anos_emprego, carro, casa, tipo_moradia):
    motivos = []
    RENDA_MINIMA_ANUAL = 7_200 # R$ 600/mes -- minimo absoluto
    RENDA_PER_CAPITA_MIN = 3_600 # R$ 300/mes por membro da familia
    RENDA_ESTUDANTE_MIN = 18_000 # R$ 1.500/mes -- estudante sem fiador
    RENDA_DATASET_MIN = 27_000 # minimo observado nos dados de treino
    renda_per_capita = rendimento / max(membros_familia, 1)

    if rendimento < RENDA_MINIMA_ANUAL:
        motivos.append('Renda abaixo do minimo exigido de R$ 7.200/ano.')
    if renda_per_capita < RENDA_PER_CAPITA_MIN:
        motivos.append('Renda per capita familiar insuficiente.')
    if categoria_renda == 'Estudante' and rendimento < RENDA_ESTUDANTE_MIN:
        motivos.append('Estudantes precisam comprovar renda minima de R$ 1.500/mes.')

    aprovado = len(motivos) == 0
    return aprovado, motivos, aviso_fora_distribuicao
```

app.py

Camada 2 — Predição ML e exibição do resultado

```
if st.button('Enviar'):  
    model = joblib.load('modelo/xgb.joblib')  
    prob = model.predict_proba(cliente_pred)  
    prob_mau = prob[-1][1]  
    prob_bom = prob[-1][0]  
  
    col1, col2 = st.columns(2)  
    col1.metric('Risco de inadimplencia', f'{prob_mau * 100:.1f}%')  
    col2.metric('Perfil positivo', f'{prob_bom * 100:.1f}%')  
    st.progress(float(prob_mau))  
  
    if prob_mau < threshold:  
        st.success('Cartao de credito aprovado!')  
        st.balloons()  
    else:  
        st.error('Credito nao liberado.')
```

utils.py — Transformers Customizados do Pipeline

utils.py

DropFeatures e MinMaxWithFeatNames

```

class DropFeatures(BaseEstimator, TransformerMixin):
    def __init__(self, feature_to_drop=['ID_Cliente']):
        self.feature_to_drop = feature_to_drop
    def fit(self, df):
        return self
    def transform(self, df):
        if set(self.feature_to_drop).issubset(df.columns):
            df.drop(self.feature_to_drop, axis=1, inplace=True)
        return df

class MinMaxWithFeatNames(BaseEstimator, TransformerMixin):
    def __init__(self, min_max_scaler_ft=[
        'Idade', 'Rendimento_anual', 'Tamanho_familia', 'Anos_empregado',
        'Renda_per_capita', 'Score_patrimonio', 'Score_contatos',
        'Renda_por_ano_emprego']):
        self.min_max_scaler_ft = min_max_scaler_ft
    def fit(self, df):
        return self
    def transform(self, df):
        cols = [c for c in self.min_max_scaler_ft if c in df.columns]
        if cols:
            df[cols] = MinMaxScaler().fit_transform(df[cols])
        return df

```

retrain_model.py — Script de Retreinamento

retrain_model.py

Engenharia de features derivadas

```

def criar_features_derivadas(df):
    df = df.copy()
    df['Renda_per_capita'] = df['Rendimento_anual'] / df['Tamanho_familia'].clip(lower=1)
    df['Score_patrimonio'] = df['Tem_carro'] + df['Tem_casa_propria']
    df['Score_contatos'] = (df['Tem_telefone_trabalho']
                           + df['Tem_telefone_fixo'] + df['Tem_email'])
    df['Renda_por_ano_emprego'] = df['Rendimento_anual'] / df['Anos_empregado'].clip(lower=0.5)
    return df

```

retrain_model.py

Treino, avaliação e persistência do modelo

```

modelo = GradientBoostingClassifier(
    n_estimators=300, learning_rate=0.05,
    max_depth=4, min_samples_leaf=20,
    subsample=0.8, random_state=SEED,
)
modelo.fit(X_train, y_train)

probs = modelo.predict_proba(X_test)[: , 1]
auc = roc_auc_score(y_test, probs)
print(f'AUC no teste: {auc:.4f}')

joblib.dump(modelo, 'modelo/xgb.joblib')
print('Modelo salvo em modelo/xgb.joblib')

```